TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 9: Watching System Calls with strace

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Part 1: See the calls of a real command **5**

1.1 Running `strace` on `ls -l` 5

1.2 Observations for Part 1 7

2 Part 2: A program that calls the kernel directly **8**

2.1 Source code and execution of `syscalls.c` 8

2.2 Tracing `./syscalls` with `strace` 9

2.3 Observations for Part 2 9

3 Part 3: Count one kind of call **10**

3.1 Counting individual system call types with `strace -e trace=... | grep -c` 10

3.2 Summary profiling per system call with `strace -c` 10

3.3 Observations for Part 3 12

Conclusions **13**

ABSTRACT

The goal of this laboratory is to observe and analyze the system-call boundary between user-space processes and the Linux kernel using `strace`. In Part 1, `strace` is used to trace `ls -l` while filtering `openat`, `read`, `write`, and `close`, identifying how standard utilities load shared libraries, read directory and user metadata, and write output to file descriptor 1. In Part 2, a C program (`syscalls.c`) directly invokes `open`, `write`, `read`, and `close` without `stdio` file buffering, demonstrating kernel file-descriptor allocation starting at descriptor 3. In Part 3, individual system call types are counted and compared using both single-call filtering (`strace -e trace=... | grep -c`) and kernel-call summary profiling (`strace -c`).

Keywords: Linux, system calls, strace, openat, read, write, close, file descriptors, syscall profiling

INTRODUCTION

Goal of the laboratory (Scopul lucrării)

The purpose of this laboratory work is to cross and inspect the boundary between user-space programs and the operating system kernel—the system call interface—on a real Linux system using `strace`. Specifically, the objectives are:

1. **Part 1 (See the calls of a real command):** Run `strace` on `ls -l` to observe the sequence of `openat`, `read`, `write`, and `close` system calls performed during program startup, directory reading, and terminal output, and identify which system call and file descriptor `ls` uses to print to the screen.
2. **Part 2 (A program that calls the kernel directly):** Write, compile, and trace a C program (`syscalls.c`) that creates a file, writes a string to it, and reads it back using only the `open`, `write`, `read`, and `close` system calls, and explain why the kernel assigns file descriptor 3.
3. **Part 3 (Count one kind of call):** Count how many times a program invokes a single kind of system call using both `strace -e trace=<syscall> ... | grep -c` and `strace -c`, and explain how output buffering and failed file probes affect individual call counts.

All experiments were carried out on Linux (openSUSE Tumbleweed, x86_64) using GCC with `-Wall -Wextra`. Complete trace logs and command outputs are recorded in `evidence/` and `report.md`.

1 PART 1: SEE THE CALLS OF A REAL COMMAND

1.1 Running `strace` on `ls -l`

To observe how a standard command interacts with the kernel, `ls -l` was executed under `strace` with the filter `-e trace=openat,read,write,close`:

```
$ strace -f -e trace=openat,read,write,close ls -l
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib64/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 1024) = 1024
close(3)                                = 0
openat(AT_FDCWD, "/lib64/libcap.so.2", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\300\214\0\0\0\0\0\0"..., 1024) = 1024
close(3)                                = 0
openat(AT_FDCWD, "/lib64/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\256\267\2\0\0\0\0\0"..., 1024) = 1024
close(3)                                = 0
openat(AT_FDCWD, "/lib64/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 1024) = 1024
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file
    or directory)
openat(AT_FDCWD, "/usr/share/locale/locale.alias", O_RDONLY|O_CLOEXEC) = 3
read(3, "# Locale name alias data base.\n#"..., 4096) = 2940
read(3, "", 4096)                       = 0
close(3)                                = 0
...
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
close(3)                                = 0
openat(AT_FDCWD, "/etc/nsswitch.conf", O_RDONLY|O_CLOEXEC) = 4
read(4, "#\n# /etc/nsswitch.conf\n#\n# An ex"..., 4096) = 1221
read(4, "", 4096)                       = 0
close(4)                                = 0
openat(AT_FDCWD, "/etc/passwd", O_RDONLY|O_CLOEXEC) = 4
read(4, "root:x:0:0:root:/root:/bin/bash\n"..., 4096) = 1979
close(4)                                = 0
openat(AT_FDCWD, "/etc/group", O_RDONLY|O_CLOEXEC) = 4
read(4, "root:x:0:\nbin:x:1:daemon\ndaemon:"..., 4096) = 853
close(4)                                = 0
...
write(1, "total 416\n", 10total 416
)             = 10
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = 3
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\20\0\0\0\0"..., 4096) = 2424
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\20\0\0\0\0"..., 4096) = 1509
close(3)                                = 0
write(1, "-rw-r--r--. 1 victor victor   40"..., 57-rw-r--r--. 1 victor victor   4033 Nov  8 21:00
     lab9.tex
) = 57
write(1, "-rw-r--r--. 1 victor victor    7"..., 57-rw-r--r--. 1 victor victor    783 Nov  8 21:01
     notes.md
) = 57
write(1, "-rw-r--r--. 1 victor victor     "..., 57-rw-r--r--. 1 victor victor     14 Nov  8 13:33
     note.txt
) = 57
```

```
write(1, "-rw-r--r--. 1 victor victor 3604"..., 57-rw-r--r--. 1 victor victor 360410 Nov  8 12:42
     pic1.png
) = 57
write(1, "-rw-r--r--. 1 victor victor  211"..., 57-rw-r--r--. 1 victor victor  21159 Nov  8 13:33
     pic2.png
) = 57
write(1, "-rw-r--r--. 1 victor victor    6"..., 58-rw-r--r--. 1 victor victor    693 Nov  8 21:00
     README.md
) = 58
write(1, "-rwxr-xr-x. 1 victor victor  204"..., 57-rwxr-xr-x. 1 victor victor  20400 Nov  8 12:50
     syscalls
) = 57
write(1, "-rw-r--r--. 1 victor victor    9"..., 59-rw-r--r--. 1 victor victor    984 Nov  8 12:47
     syscalls.c
) = 59
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
```

```
close(4)                                = 0
openat(8, "lib", O_RDONLY|O_NOFOLLOW|O_CLOEXEC|O_PATH) = 4
close(8)                                = 0
openat(4, "userdb", O_RDONLY|O_NOFOLLOW|O_CLOEXEC|O_PATH) = -1 ENOENT (No such file or directory)
close(4)                                = 0
close(5)                                = 0
close(6)                                = 0
close(7)                                = 0
close(3)                                = 0
openat(AT_FDCWD, "/usr/share/locale/en_US.UTF-8/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en_US.utf8/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en_US/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en.UTF-8/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en.utf8/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
write(1, "total 0\n", 8total 0
)               = 8
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = 3
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\20\0\0\0\0"..., 4096) = 2424
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\20\0\0\0\0"..., 4096) = 1509
close(3)                                = 0
write(1, "drwxr-xr-x. 1 victor victor  312"..., 55drwxr-xr-x. 1 victor victor  312 Nov  8 12:26  Desktop
) = 55
write(1, "drwxr-xr-x. 1 victor victor   20"..., 57drwxr-xr-x. 1 victor victor   20 Sep  8 11:16  Documents
) = 57
write(1, "drwxr-xr-x. 1 victor victor 2598"..., 57drwxr-xr-x. 1 victor victor 2598 Oct 28 15:42  Downloads
) = 57
write(1, "drwxr-xr-x. 1 victor victor    0"..., 55drwxr-xr-x. 1 victor victor    0 Sep  9 17:59  install
) = 55
write(1, "drwxr-xr-x. 1 victor victor   34"..., 62drwxr-xr-x. 1 victor victor   34 Sep 14 15:17  kwallet_backup
) = 62
write(1, "drwxr-xr-x. 1 victor victor    0"..., 53drwxr-xr-x. 1 victor victor    0 Sep  1 16:29  Music
) = 53
write(1, "drwxr-xr-x. 1 victor victor  272"..., 57drwxr-xr-x. 1 victor victor  272 Nov  8  2026  Nextcloud
) = 57
write(1, "drwxr-xr-x. 1 victor victor   86"..., 62drwxr-xr-x. 1 victor victor   86 Sep 17 12:34  pdf-tools-venv
) = 62
write(1, "drwxr-xr-x. 1 victor victor   22"..., 56drwxr-xr-x. 1 victor victor   22 Sep  4 13:51  Pictures
) = 56
write(1, "drwxr-xr-x. 1 victor victor  174"..., 56drwxr-xr-x. 1 victor victor  174 Nov  4 18:59  Projects
) = 56
write(1, "drwxr-xr-x. 1 victor victor    0"..., 54drwxr-xr-x. 1 victor victor    0 Sep  1 16:29  Public
) = 54
write(1, "drwxr-xr-x. 1 victor victor   52"..., 61drwxr-xr-x. 1 victor victor   52 Oct  6 10:06  RiderProjects
) = 61
write(1, "drwxr-xr-x. 1 victor victor   48"..., 54drwxr-xr-x. 1 victor victor   48 Oct 12 10:10  shtest
) = 54
write(1, "drwxr-xr-x. 1 victor victor    0"..., 57drwxr-xr-x. 1 victor victor    0 Sep  1 16:29  Templates
) = 57
write(1, "drwxr-xr-x. 1 victor victor    0"..., 54drwxr-xr-x. 1 victor victor    0 Sep  1 16:29  Videos
) = 54
write(1, "drwxr-xr-x. 1 victor victor   32"..., 63drwxr-xr-x. 1 victor victor   32 Sep 28 17:12 'VirtualBox VMs'
) = 63
write(1, "drwxr-xr-x. 1 victor victor  264"..., 57drwxr-xr-x. 1 victor victor  264 Oct 12 10:18  xv6-riscv
) = 57
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
victor@localhost:~> strace -f -e trace=openat,read,write,close ls -l 2>&1 > /dev/null | wc -l
187
victor@localhost:~>
```

Figure 1.1 – Terminal trace of `strace -f -e trace=openat,read,write,close ls -l`

1.2 Observations for Part 1

1. **Which system call does `ls` use to print to the screen? To which descriptor does it write?** `ls` uses the `write` system call (`write(1, ..., ...)`) to print the formatted directory listing to the screen. It writes to **file descriptor 1**, which is **standard output (`stdout`)**. When `stdout` is connected to a terminal, the C standard library uses line buffering, so each line of the `ls -l` listing is emitted via a separate `write(1, ...)` call.
2. **How many system calls does one simple `ls -l` make, roughly, and what happens during the trace?** Even for a single directory, `ls -l` makes roughly **174–186 calls** among the four filtered types (`openat`, `read`, `write`, `close`) and **368–371 total system calls** when all system call types are included. The trace unfolds in four phases:
    - *Dynamic linker setup:* opening and reading ELF headers from `/etc/ld.so.cache`, `libselinux.so.1`, `libcap.so.2`, `libc.so.6`, and `libpcre2-8.so.0`;
    - *Locale configuration:* opening `LC_*` locale files and probing optional message catalogs (`coreutils.mo`), where missing optional catalogs return `-1 ENOENT`;
    - *Directory and account lookup:* opening the target directory `"."` with `O_DIRECTORY`, reading `/etc/nsswitch.conf`, `/etc/passwd`, `/etc/group`, and `/etc/localtime` to convert numeric UIDs, GIDs, and timestamps into human-readable strings;
    - *Output and cleanup:* writing each line to descriptor 1 with `write(1, ...)` and closing descriptors 1 and 2 before exiting.

2 PART 2: A PROGRAM THAT CALLS THE KERNEL DIRECTLY

2.1 Source code and execution of `syscalls.c`

The program `syscalls.c` creates `note.txt`, writes a 14-byte string (`"hello, kernel\n"`) directly through the kernel using `write`, closes the descriptor, reopens `note.txt` read-only, and reads the bytes back using `read`:

```
#include <fcntl.h>   /* open */
#include <unistd.h>  /* write, read, close */
#include <string.h>  /* strlen */
#include <stdio.h>   /* printf */

int main(void) {
    const char *msg = "hello, kernel\n";

    /* open() is a system call: ask the kernel for a file, get back a descriptor */
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);

    /* write() is a system call: hand the bytes to the kernel */
    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);

    /* read it back, through the kernel again */
    fd = open("note.txt", O_RDONLY);
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);
    return 0;
}
```

Compiling and running `syscalls.c` produces the expected output:

```
$ gcc -Wall -Wextra -o syscalls syscalls.c
$ ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
```

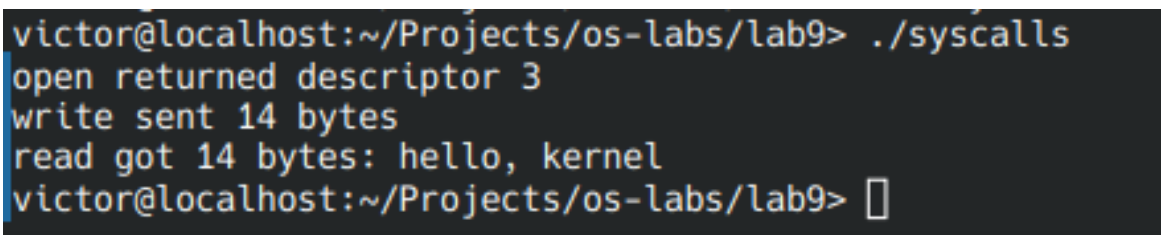


Figure 2.1 – Execution of the direct system-call program `./syscalls`

2.2 Tracing `./syscalls` with `strace`

Running `strace -e trace=openat,write,read,close ./syscalls` shows both the dynamic loader setup and our six direct file system calls:

```
$ strace -e trace=openat,write,read,close ./syscalls
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib64/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\256\267\2\0\0\0\0\0"..., 1024) = 1024
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "open returned descriptor 3\n", 27open returned descriptor 3
) = 27
write(3, "hello, kernel\n", 14)         = 14
write(1, "write sent 14 bytes\n", 20write sent 14 bytes
)   = 20
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
write(1, "read got 14 bytes: hello, kernel"..., 33read got 14 bytes: hello, kernel
) = 33
close(3)                                = 0
+++ exited with 0 +++
```

2.3 Observations for Part 2

- **The descriptor returned by the first `openat` is 3, not 0, 1 or 2. Why are those three already taken?** Every POSIX process inherits three pre-opened standard stream descriptors from its parent shell when started:
  - Descriptor **0**: standard input (`stdin`, `STDIN_FILENO`),
  - Descriptor **1**: standard output (`stdout`, `STDOUT_FILENO`),
  - Descriptor **2**: standard error (`stderr`, `STDERR_FILENO`).

  Because the Linux kernel always allocates the **lowest-numbered unused file descriptor** in the process's file descriptor table, the first call to `open("note.txt", ...)` (implemented by glibc via the `openat` system call relative to `AT_FDCWD`) returns descriptor **3**. When `close(3)` is called after writing, descriptor 3 becomes free again, so the second `open("note.txt", O_RDONLY)` immediately reuses descriptor **3**.

3 PART 3: COUNT ONE KIND OF CALL

3.1 Counting individual system call types with `strace -e trace=... | grep -c`

Whereas piping a multi-call trace to `wc -l` counts all displayed lines combined (including the `+++ exited with 0 +++` status line), counting a **single kind of system call** requires either filtering a single system call with `-e trace=<syscall>` and counting matching lines with `grep -c`, or using `strace -c`.

Running `strace -e trace=<syscall>` with `grep -c` on `ls -l` for each of the four system calls individually:

```
$ strace -e trace=openat ls -l 2>&1 >/dev/null | grep -c '^openat'
102
$ strace -e trace=close ls -l 2>&1 >/dev/null | grep -c '^close'
56
$ strace -e trace=read ls -l 2>&1 >/dev/null | grep -c '^read'
15
$ strace -e trace=write ls -l 2>&1 >/dev/null | grep -c '^write'
1
```

Doing the same single-call count on `./syscalls`:

```
$ strace -e trace=openat ./syscalls 2>&1 >/dev/null | grep -c '^openat'
4
$ strace -e trace=close ./syscalls 2>&1 >/dev/null | grep -c '^close'
4
$ strace -e trace=read ./syscalls 2>&1 >/dev/null | grep -c '^read'
2
$ strace -e trace=write ./syscalls 2>&1 >/dev/null | grep -c '^write'
2
```

In `./syscalls`, the 4 `openat` and 4 `close` calls consist of 2 library loads (`/etc/ld.so.cache` and `libc.so.6`) plus our 2 opens/closes of `note.txt`. The 2 `read` calls are 1 ELF header read of `libc.so.6` plus our `read(3, ...)` from `note.txt`. The 2 `write` calls are our unbuffered `write(3, "hello, kernel\n", 14)` plus 1 buffered `write(1, ...)` flushing all three `printf` lines at exit when `stdout` is redirected to `/dev/null`.

3.2 Summary profiling per system call with `strace -c`

The built-in `strace -c` option counts invocations, errors, and kernel execution time for each system call type individually. Restricting `strace -c` to the four calls from Part 1:

```
$ strace -c -e trace=openat,read,write,close ls -l >/dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 72.06    0.000539           5       102        48 openat
 21.66    0.000162           2        56           close
  6.02    0.000045           3        15           read
  0.27    0.000002           2         1           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000748           4       174        48 total
```

Running `strace -c ls -l >/dev/null` without a filter counts every distinct system call type made by `ls -l`:

```
$ strace -c ls -l >/dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 34.95    0.001475          14       102        48 openat
 14.31    0.000604          14        43           mmap
  8.70    0.000367         367         1           execve
  7.68    0.000324           5        56           close
  5.71    0.000241           6        35           statx
  4.83    0.000204          10        20           newfstatat
  4.15    0.000175          21         8           mprotect
  4.03    0.000170           5        31           fstat
  3.15    0.000133           8        15           read
  2.30    0.000097          10         9           llistxattr
  1.82    0.000077          25         3         2 access
  1.54    0.000065           7         9           lgetxattr
  0.90    0.000038           4         8           lseek
  0.81    0.000034           5         6         4 prctl
  0.78    0.000033          16         2           getdents64
  0.76    0.000032          16         2         2 ioctl
  0.76    0.000032          16         2           statfs
  0.55    0.000023           5         4           futex
  0.50    0.000021           7         3           brk
  0.45    0.000019           4         4           rt_sigprocmask
  0.40    0.000017           8         2           getrandom
  0.36    0.000015          15         1           prlimit64
  0.14    0.000006           6         1           arch_prctl
  0.14    0.000006           6         1           set_tid_address
  0.14    0.000006           6         1           set_robust_list
  0.14    0.000006           6         1           rseq
  0.00    0.000000           0         1           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.004220          11       371        56 total
```

```
victor@localhost:~/Projects/os-labs/lab9> strace -e trace=openat ls -l 2>&1 >/dev/null | grep -c '^openat'
102
victor@localhost:~/Projects/os-labs/lab9> strace -e trace=close ls -l 2>&1 >/dev/null | grep -c '^close'
56
victor@localhost:~/Projects/os-labs/lab9> strace -e trace=read ls -l 2>&1 >/dev/null | grep -c '^read'
15
victor@localhost:~/Projects/os-labs/lab9> strace -e trace=write ls -l 2>&1 >/dev/null | grep -c '^write'
1
victor@localhost:~/Projects/os-labs/lab9> strace -c -e trace=openat,read,write,close ls -l >/dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 72.06    0.000539           5       102        48 openat
 21.66    0.000162           2        56           close
  6.02    0.000045           3        15           read
  0.27    0.000002           2         1           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000748           4       174        48 total
```

Figure 3.1 – Counting individual system call types with `strace -e trace=... | grep -c` and `strace -c`

3.3 Observations for Part 3

1. **Matching `openat` and `close` counts:** Out of **102** `openat` calls, **48** return `-1 ENOENT` (failed lookups for optional locale catalogs and userdb sockets), leaving $102 - 48 = 54$ successful opens that allocate file descriptors. Adding the closing of standard output (`close(1)`) and standard error (`close(2)`) before exit yields $54 + 2 = \mathbf{56}$ `close` calls.
2. **Effect of standard output buffering on `write` count:** When `ls -l` outputs directly to the terminal (Part 1), `glibc` detects a terminal via `ioctl(1, TCGETS, ...)` and configures `stdout` as line-buffered, resulting in one `write(1, ...)` call per output line (9 `write` calls for 8 files plus the `total` line). When `stdout` is redirected to `/dev/null`, `ioctl` returns `ENOTTY` and `glibc` switches to a 4096-byte block buffer, coalescing all output lines into a single `write(1, ..., 469)` call.

CONCLUSIONS

Using `strace`, this laboratory observed the Linux system-call interface from three perspectives:

- In **Part 1**, tracing `ls -l` showed how a real utility loads shared libraries, queries locale and account databases, and prints to the screen via `write` on file descriptor 1 (`stdout`).
- In **Part 2**, writing and tracing `syscalls.c` confirmed that `open`, `write`, `read`, and `close` cross directly into the kernel and that the first opened file receives descriptor 3 because descriptors 0, 1, and 2 are already occupied by `stdin`, `stdout`, and `stderr`.
- In **Part 3**, counting single system call types with `strace -e trace=<call> ... | grep -c` and `strace -c` quantified the exact per-call breakdown for `ls -l` (102 `openat`, 56 `close`, 15 `read`, and 1 block-buffered `write` out of 371 total system calls) and explained the relationship between successful `openat` calls, `close` calls, and `stdio` buffering.